

- MASTG
- Prevent Screenshots in SurfaceViews

MASTG-TEST-0294: `SecureOn` Not Used to Prevent Screenshots in Compose Dialogs

MASTG-TEST-0315: Sensitive Data Exposed via Notifications

MASTG-TEST-0316: App Exposing User Authentication Data in Text Input Fields

MASTG-TEST-0320: WebViews Not Cleaning Up Sensitive Data

MASTG-TEST-0334: Native Code Exposed Through WebViews

MASTG-TEST-0340: References to Overlay Attack Protections

L2

ANDROID

MASWE-0053

TEST

MASTG-TEST-0316: App Exposing User Authentication Data in Text Input Fields

Overview

This test verifies that the app handles user input correctly, ensuring that access codes (passwords or pins) and verification codes (OTPs) are not exposed in plain text within text input fields.

Proper masking (e.g., dots instead of input characters) of these codes is essential to protect user privacy. This can be achieved by using appropriate input types that obscure the characters entered by the user. In Jetpack Compose, `SecureTextField` uses `TextObfuscationMode`, which by default is `TextObfuscationMode.RevealLastTyped`, so a developer can simply use `SecureTextField` without explicitly setting `textObfuscationMode` unless another behavior is required.

XML view:

```
<EditText
    android:inputType="textPassword"
    ...
/>
```

Jetpack Compose:

```
SecureTextField(
    // textObfuscationMode defaults to TextObfuscationMode.RevealLastTyped
    textObfuscationMode = TextObfuscationMode.RevealLastTyped, // or TextObfuscationMode.Hidden
    ...
)
```

Note

Even if `SecureTextField` uses the default `TextObfuscationMode.RevealLastTyped` or is configured explicitly with `RevealLastTyped` or `Hidden`, it can later be changed to `Visible` programmatically.

Steps

- Use [Reverse Engineering Android Apps](#) to reverse engineer the app.
- Use [Static Analysis on Android](#) to look for references to the text field classes and text obfuscation APIs.
- Manually evaluate and shortlist the fields for access or verification codes usage.

Observation

The output should contain a list of locations where text input fields for access or verification codes are used.

Evaluation

The test case fails if any text input field used for access or verification codes is found to be unmasked. For example, due to the following:

- `TextField` is used
- `SecureTextField` is used but configured with `TextObfuscationMode.Visible`

Note

This test may produce false negatives if the app uses custom text input controls that do not rely on standard classes such as `TextField` or `SecureTextField` (for example in custom UI frameworks or game engines).

Demos

 MASTG-DEMO-0079: App Exposing Access and Verification Codes in Text Input Fields